

Python 응용: API 활용 국내 여행지 추천 프로그램 개발 최종 제출 보고서

— LLM API(Gemini) × 지도/장소 검색 API(Kakao Local) 연동 —

1. 미션 개요

현대 서비스 대부분은 API를 통해 데이터를 주고받는다. 본 미션은 단일 API 호출을 넘어, LLM API와 지도/장소 검색 API를 조합하여 하나의 인사이트(여행 리포트)를 만들어내는 흐름을 직접 구현하는 것을 목표로 한다.

사용자가 여행 날짜를 입력하면

- ① Gemini API가 해당 시기에 여행하기 좋은 국내 지역을 추천하고,
- ② Kakao Local API가 추천 지역의 맛집 정보를 검색하며,
- ③ 다시 Gemini API가 두 결과를 종합하여 최종 여행 리포트를 Markdown으로 생성한다.

1-1. 개발 환경

항목	요구 내용	구현 위치 / 상태
개발 언어	Python 3.10 이상	
개발 도구	VSCode, Git	
LLM API	Google Gemini API (gemini-3.5-flash-lite)	llm_client.py
지도/장소 검색 API	Kakao Local API (키워드 기반 장소 검색)	map_client.py

2. 과제 목표 달성 체크리스트

미션 문서에서 요구한 4가지 학습 목표에 대해, 본 프로젝트를 통해 아래와 같이 설명 가능한 수준까지 학습 및 구현을 완료하였다.

항목	요구 내용	구현 위치 / 상태
REST API 구조 이해	요청/응답 구조와 GET/POST 메서드 차이 설명 가능	완료
구조화 출력 활용	LLM 출력(JSON)을 다음 단계(지도 검색) 입력으로 연결	완료
오류 대응 원칙	인증(401/403)/쿼터/네트워크/파싱 오류 대응 원칙 체득	완료
.env 키 관리	API 키를 코드에 직접 작성하지 않는 이유 및 방법 습득	완료

3. 기능 요구사항 이행 현황

항목	요구 내용	구현 위치 / 상태
CLI 인터페이스	argparse, (표준라이브러리)필수 옵션 --date, 날짜 형식 검증	travel_planner.py / parse_args()
API 제공자 선택	Gemini + Kakao Local , 최소 응답 필드 확보	llm_client.py, map_client.py
.gitignore / README	불필요 파일 제외, 프로젝트 제목 작성	.gitignore, README.md
LLM 1차 추천 연동	JSON 스키마(city/weather/events/reason) 준수	get_initial_recommendation()

항목	요구 내용	구현 위치 / 상태
지도 API 맛집 검색	5곳 검색, 0건 시 중단 없이 다음 단계 진행	search_restaurants()
LLM 최종 리포트 생성	지역/이유/날씨/행사/맛집/일정 포함 Markdown	generate_final_report()
오류 처리(try-except)	키 미설정 종료, API 실패 시 데이터없음 처리, 재시도 1회	check_api_keys(), 각 클라이언트
API 키 관리(보안)	.env 사용(환경변수), 코드/제출물에 키 미노출.	.env, .gitignore
결과 저장	results/ 폴더에 원본 JSON + 리포트 MD 저장	cache_manager.save_cache()

4. 결과물 1 — CLI 기반 Python 프로그램

4-1. 프로젝트 구조

```

ravel_planner/
├── travel_planner.py      # 메인 CLI 프로그램 (진입점)
├── llm_client.py          # Gemini API 연동 (1차 추천 + 리포트 생성)
├── map_client.py          # Kakao Local API 연동 (맛집 검색)
├── cache_manager.py       # 결과 캐싱 (보너스 2)
├── .env                  # API 키 (Git 미포함)
├── .env.example           # 키 형식 예시 (Git 포함)
├── .gitignore             # .env, venv 등 제외
├── README.md             # 프로젝트 설명서
└── results/              # 실행 결과 저장 폴더

```

4-2. 실행 방법

```
python travel_planner.py --date "2026-03-15"
```

4-3. 실행 화면 (CLI 골격 – 날짜 검증 테스트)

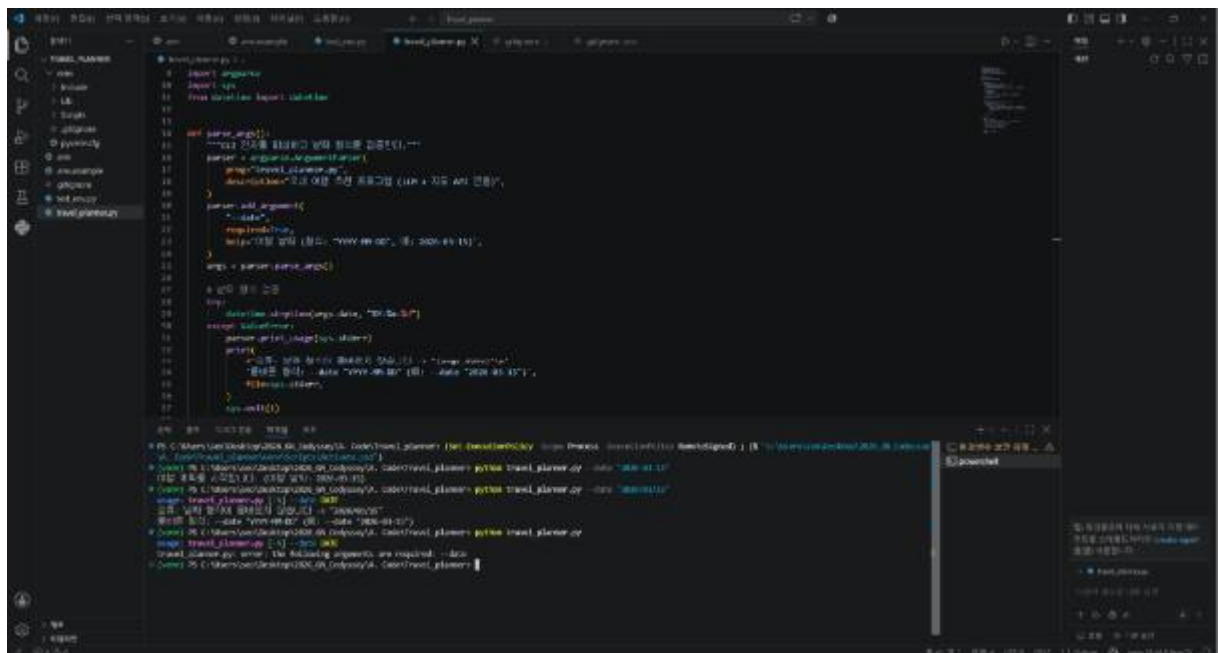


그림 1. argparse 기반 CLI 골격 및 날짜 형식 검증 테스트 결과 (STEP 4)

4-4. 핵심 로직 요약 — 날짜 검증

```
try:
    datetime.strptime(args.date, "%Y-%m-%d")
except ValueError:
    parser.print_usage(sys.stderr)
    print(f'오류: 날짜 형식이 올바르지 않습니다 -> "{args.date}"')
    sys.exit(1)
```

4-5. 핵심 로직 요약 — API 키 미설정 시 즉시 종료

```
def check_api_keys():
    missing = []
    if not os.getenv("GEMINI_API_KEY"): missing.append("GEMINI_API_KEY")
    if not os.getenv("KAKAO_REST_API_KEY"): missing.append("KAKAO_REST_API_KEY")
    if missing:
        print(" 다음 API 키가 설정되지 않았습니다:", ", ".join(missing))
        sys.exit(1)
```

5. 결과물 2 — 실행 결과 데이터 (results/)

실행 시 results/ 폴더에 실행 날짜를 기준으로 원본 데이터 JSON과 최종 리포트 Markdown이 자동 생성된다.

5-1. 원본 데이터 JSON 구조 (results/{날짜}_raw_data.json)

```
{
  "date": "2026-05-20",
  "recommendations": [
    {
      "city": "경주", "weather": "...", "events": [...],
      "reason": "...",
      "restaurants": [
        {
          "name": "...", "address": "...", "category": "...",
          "url": "...", "x": "...", "y": "...", ...
        }
      ]
    },
    { "city": "부산", ... }, { "city": "남해", ... }
  ],
  "errors": []
}
```

5-2. 최종 리포트 Markdown 구조 (results/{날짜}_travel_plan.md)

```
# 2026-05-20 국내 여행 추천 리포트
## 추천 지역
## 추천 이유
## 날씨 요약
## 행사/축제
## 맛집 추천
## (선택) 1일 일정 제안
## 오류 요약(errors)
```

5-3. 실행 화면 (LLM 1차 추천 + Kakao 맛집 검색 정상 동작)

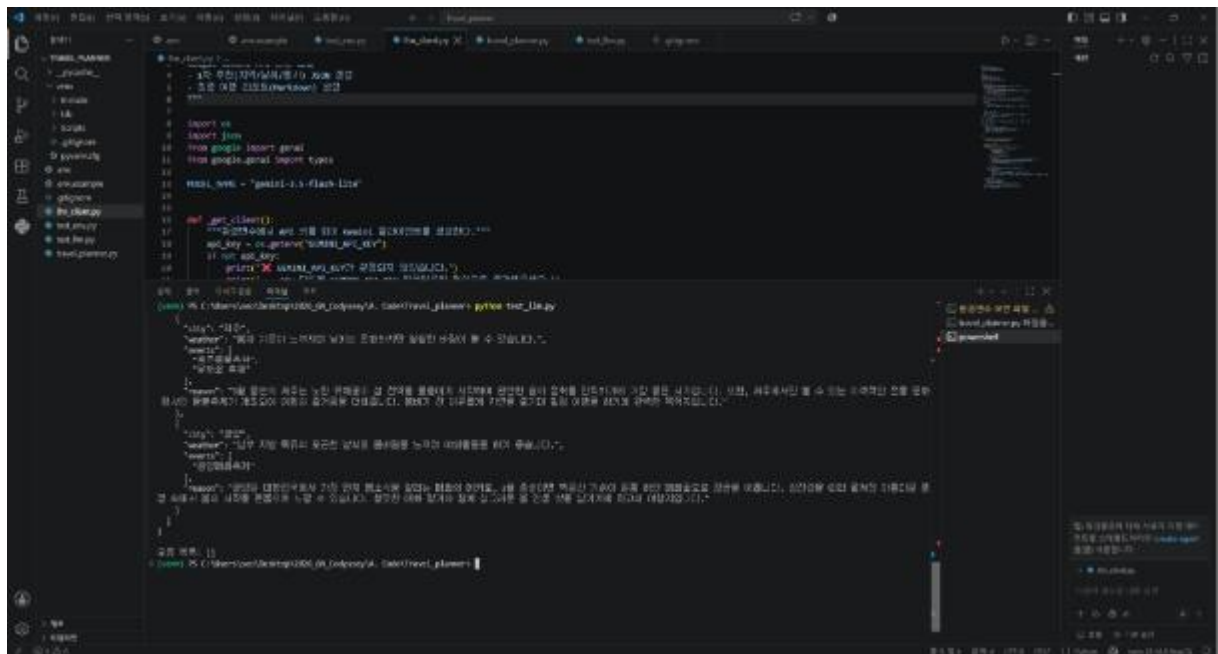


그림 2. Gemini API 1차 추천(제주/광양) 및 llm_client.py 정상 응답 확인 (STEP 5)

5-4. 전체 통합 실행 결과 로그 (STEP 11 통합 테스트)

```
$ python travel_planner.py --date "2026-05-20"
여행 계획을 시작합니다. (여행 날짜: 2026-05-20)
[1/3] 1차 추천 생성 중(LLM)...
- recommended_city: "경주"
- recommended_city: "부산"
- recommended_city: "남해"
[2/3] 맛집 검색 중(지도/장소 API)...
- '경주' 맛집 5곳 검색 완료
- '부산' 맛집 5곳 검색 완료
- '남해' 맛집 5곳 검색 완료
원본 데이터 저장 완료: results\2026-05-20_raw_data.json
[3/3] 최종 리포트 생성 중(LLM)...
- 리포트 생성 완료

완료! results\2026-05-20_travel_plan.md 를 확인하세요.
```

5-5. 보너스 2 — 캐싱 재실행 결과 (API 재호출 없음)

```
$ python travel_planner.py --date "2026-05-20"
여행 계획을 시작합니다. (여행 날짜: 2026-05-20)
[캐시 발견: results\2026-05-20_raw_data.json]
기존 원본 데이터를 재사용합니다. (Gemini/Kakao API 호출을 건너뛰니다)
[3/3] 최종 리포트 생성 중(LLM)...
- 리포트 생성 완료
```

6. 결과물 3 — README.md

README.md에는 아래 4가지 필수 항목을 모두 포함하였다.

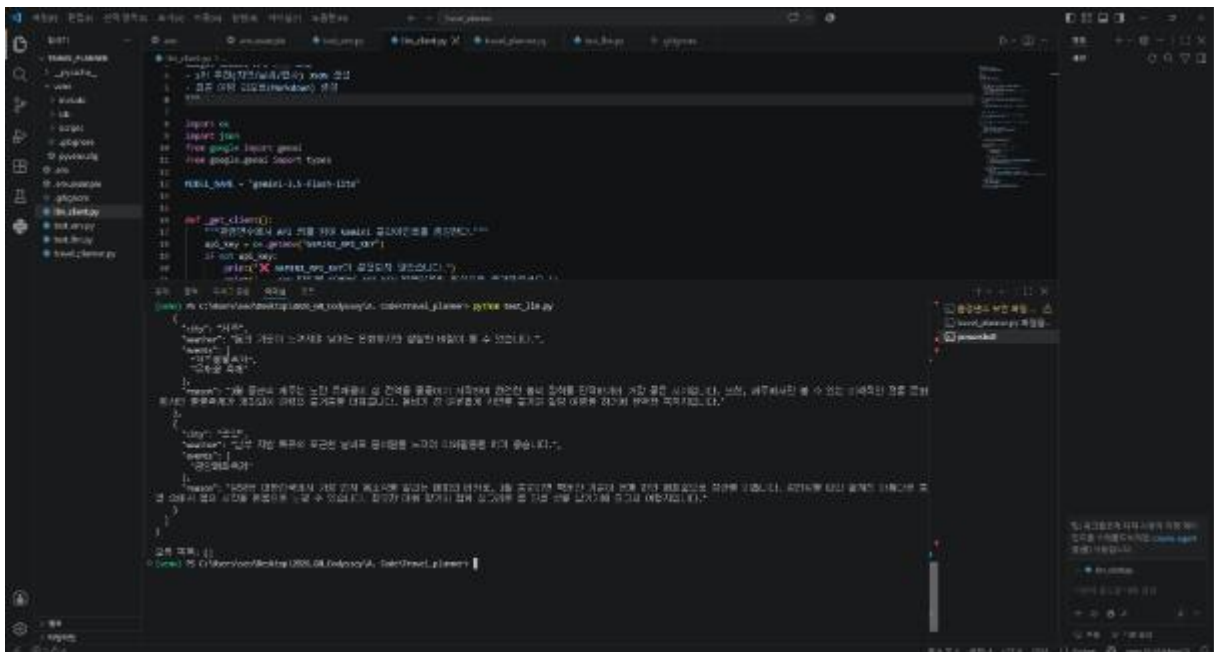
항목	요구 내용	구현 위치 / 상태
프로그램 개요	3단계 처리 흐름(1차 추천 → 맛집 검색 → 리포트 생성) 설명	README.md #1
실행 방법	가상환경 생성, 패키지 설치, 실행 커맨드 안내	README.md #3
API 키 설정 방법	Gemini/Kakao 키 발급처, .env 작성법 안내	README.md #4
결과물 확인 방법	results/ 폴더의 JSON/MD 파일 설명	README.md #5
보안 주의사항	키 미노출 원칙, .gitignore 등록, 키 재발급 안내	README.md #7

7. 보너스 과제 구현 내역 (2개 모두 진행)

7-1. 보너스 1 — 복수 지역 추천

1차 추천 JSON의 스키마를 recommendations 배열로 확장하여, Gemini가 한 번의 호출로 2~3개 지역을 동시에 추천하도록 프롬프트를 설계하였다. 이후 각 지역에 대해 반복문으로 맛집을 검색하고, 최종 리포트에서도 지역별 섹션으로 정리된다.

- 포인트: 반복 처리(for문) + API 요청 관리 + 다중 결과 구조 설계 경험
- 결과: 2026-05-20 실행 시 경주 / 부산 / 남해 3개 지역이 리포트에 함께 정리됨

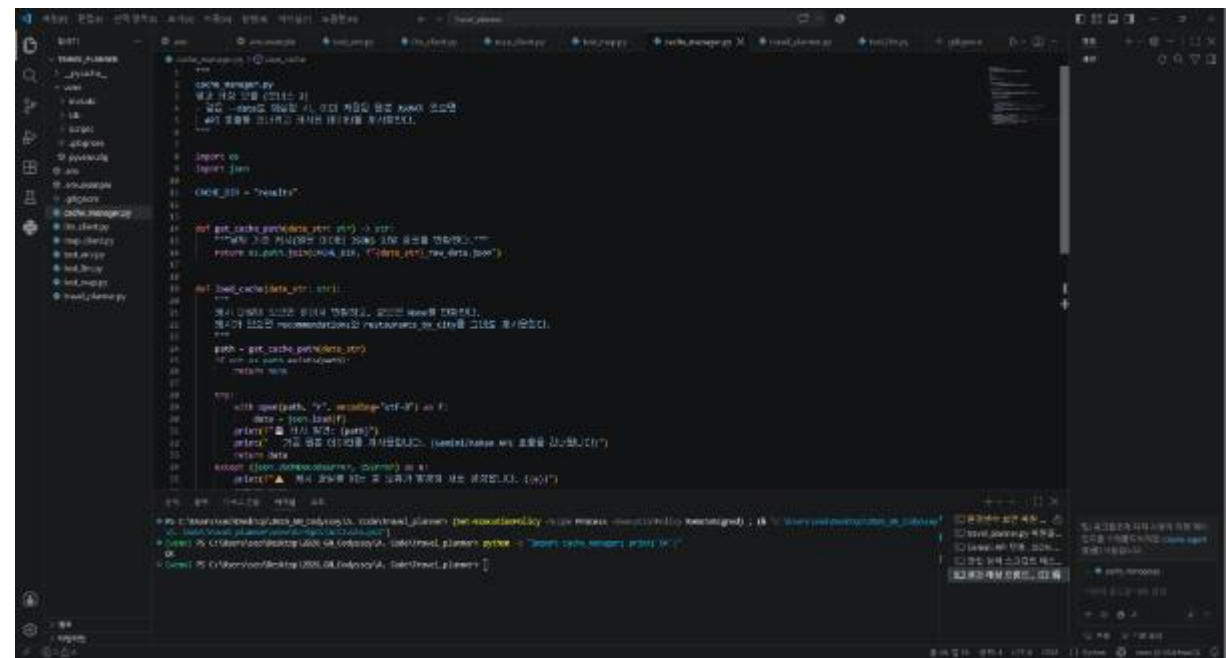


7-2. 보너스 2 — 결과 캐싱

cache_manager.py를 구현하여, 동일한 --date로 재실행 시 results/{날짜}_raw_data.json이 이미 존재하면 Gemini(1차 추천)와 Kakao(맛집 검색) API 호출을 모두 건너뛰고, 저장된 원본 데이터를 재사용

해 리포트만 재생성하도록 하였다.

- 포인트: 외부 API 비용/속도 최적화의 기본 원리 체득
- 결과: STEP 11 재실행 테스트에서 '🗺 캐시 발견' 메시지와 함께 API 재호출 없이 완료됨



8. 트러블슈팅 기록

개발 과정에서 발생한 주요 오류와 해결 과정을 기록한다. 이는 미션 목표 중 '외부 API 호출에서 발생하는 대표 오류와 대응 원칙 설명'과 직접 연결되는 실전 경험이다.

① Gemini 모델 404 NOT_FOUND

- 증상: gemini-2.5-flash 모델 호출 시 "This model ... available to new users" 404 오류 발생.
- 원인: Gemini 모델 세대가 3.x로 전환되며 구버전 모델이 신규 사용자에게 차단됨.
- 해결: 모델명을 gemini-3.5-flash-lite(2026년 8월 기준 GA, 저비용-저지연 모델)로 교체하여 해결.

② Kakao Local API 403 Forbidden

- 증상: 맛집 검색 요청 시 403 Forbidden, 응답 "disabled OPEN_MAP_AND_LOCAL service" 메시지.
- 원인: 카카오 개발자 콘솔에서 해당 앱에 '카카오맵(로컬)' 서비스 자체가 활성화되지 않은 상태였음.
- 해결: 콘솔에서 카카오맵 서비스 활성화 스위치 ON → 200 정상 응답 확인.

③ API 키 관리 오류 정책 구현

항목	요구 내용	구현 위치 / 상태
키 미설정	즉시 종료 + 설정 방법 안내 출력	check_api_keys()
지도 API 인증 실패	맛집=데이터 없음 처리 후 리포트 생성 진행	search_restaurants()
검색 결과 0건	중단 없이 데이터 없음으로 다음 단계 진행	search_restaurants()
LLM JSON 파싱 실패	재시도 1회, 실패 시 errors 배열에 기록	get_initial_recommendation()

9. 최종 결론 및 학습 성과

본 미션을 통해 단일 API 호출을 넘어, LLM API의 구조화된 출력을 다음 단계 API의 입력으로 연결하는 '체이닝(chaining)' 패턴을 직접 구현하였다. 이 과정에서 각 API의 인증 방식 차이(Gemini의 API 키 헤더 vs Kakao의 KakaoAK 헤더), 오류 유형별 대응 원칙(인증/쿼터/네트워크/파싱), 그리고 .env를 통한 키 관리의 실질적 필요성을 실제 트러블슈팅(403 Forbidden, 404 모델 만료)을 통해 체득하였다.

특히 Kakao Local API의 403 오류는 '키는 정상 발급되었지만 서비스가 비활성화된' 사례로, 인증 오류가 반드시 키 값 자체의 문제만은 아니라는 점을 실전에서 확인할 수 있었던 유의미한 경험이었다.

보너스 과제인 복수 지역 추천과 결과 캐싱을 모두 구현함으로써, 반복 처리 기반의 데이터 구조 설계와 외부 API 비용/속도 최적화라는 두 가지 배움 포인트를 추가로 확보하였다.

9-1. 최종 요구사항 이행 요약

항목	상태
CLI 기반 Python 프로그램 (argparse, 날짜 검증)	완료
결과 데이터 저장 (results/ JSON + Markdown)	완료
README.md (개요/실행법/키 설정/결과 확인/보안)	완료
LLM API 연동 - 1차 추천 (Gemini)	완료
지도/장소 검색 API 연동 - 맛집 검색 (Kakao Local)	완료
LLM API 연동 - 최종 리포트 생성 (Gemini)	완료
오류 처리(try-except, 재시도 1회)	완료
API 키 보안 관리 (.env, .gitignore)	완료
보너스 1 - 복수 지역 추천	완료
보너스 2 - 결과 캐싱	완료

※ 본 보고서에는 실제 실행 화면 캡처 3점(환경변수 로딩/CLI 검증, Gemini 1차 추천 응답)을 포함하였으며, 실행 로그는 개발 과정 중 터미널 출력을 그대로 인용하였다. API 키 값은 어디에도 노출되지 않았음을 확인하였다.

< 미션 수행 진행 과정 스크린샷 >

1단계. 프로젝트 폴더 생성, Python 가상환경(venv) 생성

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\user\Desktop\2026_08_Codysey\A. Code> cd "C:\Users\user\Desktop\2026_08_Codysey\A. Code"
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code> mkdir Travel_planner

현재 위치: C:\Users\user\Desktop\2026_08_Codysey\A. Code

Mode                LastWriteTime         Length Name
----                -
d-----          2026-08-18 오후 9:14                Travel_planner

PS C:\Users\user\Desktop\2026_08_Codysey\A. Code> cd Travel_planner
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> python --version
Python 3.10.4
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> python -m venv venv
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> .\venv\Scripts\activate.ps1
(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> pip install google-genai requests python-dotenv
Collecting google-genai
  Using cached google_genai-2.17.0-py3-none-any.whl.metadata (56 kB)
Collecting requests
  Using cached requests-2.31.2-py3-none-any.whl.metadata (3.0 kB)
Collecting python-dotenv
  Using cached python_dotenv-1.2.0-py3-none-any.whl.metadata (27 kB)
Collecting anyio<4.0.0,>=4.0.0 (from google-genai)
  Using cached anyio-4.1.2-py3-none-any.whl.metadata (3.0 kB)
Collecting google-auth<3.0,>=2.25.0 (from google-genai)
  Using cached google_auth-2.36.1-py3-none-any.whl.metadata (6.0 kB)
Collecting httpx<3.0,>=2.0.1 (from google-genai)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic<3.0,>=2.12.0 (from google-genai)
  Using cached pydantic-2.12.4-py3-none-any.whl.metadata (166 kB)
Collecting tenacity<9.2.0,>=8.2.3 (from google-genai)
  Using cached tenacity-8.2.3-py3-none-any.whl.metadata (1.3 kB)
Collecting annotated-types<1.0,>=1.0.0 (from google-genai)
  Using cached annotated_types-1.0.0-py3-none-any.whl.metadata (7.0 kB)
Collecting typing-extensions<4.0,>=3.10.0 (from google-genai)
  Using cached typing_extensions-4.12.0-py3-none-any.whl.metadata (3.2 kB)
Collecting distro<2.0,>=1.9.0 (from google-genai)
  Using cached distro-2.0.0-py3-none-any.whl.metadata (6.0 kB)
Collecting sniffio (from google-genai)
  Using cached sniffio-1.3.1-py3-none-any.whl.metadata (3.0 kB)
Collecting charset-normalizer<4.0,>=3.0.0 (from google-genai)
  Using cached charset_normalizer-3.4.0-py3-none-any.whl.metadata (10.0 kB)
Collecting idna<4.0,>=3.0 (from google-genai)
  Using cached idna-3.10-py3-none-any.whl.metadata (6.0 kB)
Collecting urllib3<3.0,>=2.0.0 (from google-genai)
  Using cached urllib3-2.2.3-py3-none-any.whl.metadata (6.0 kB)

Installing collected packages: urllib3, typing-extensions, tenacity, sniffio, python-dotenv, pydantic, pydantic-core, pydantic, httpx, httpcore, http, cryptography, google-auth, google-genai, requests, anyio
Successfully installed anyio-4.1.2 cryptography-44.0.0 google-auth-2.36.1 google-genai-2.17.0 http-0.6.0 httpcore-1.0.5 httpx-0.28.1 idna-3.10 pydantic-2.12.4 pydantic-core-2.21.1 python-dotenv-1.2.0 requests-2.31.2 sniffio-1.3.1 tenacity-8.2.3 typing-extensions-4.12.0 typing-inspect-0.10.0 urllib3-2.2.3
(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> pip list
```

2단계. 필요 패키지 설치_ pip install google-genai requests python-dotenv

```
using cached urllib3-2.2.3-py3-none-any.whl (126 kB)
using cached python-dotenv-1.2.0-py3-none-any.whl (22 kB)
using cached annotated-types-0.6.0-py3-none-any.whl (13 kB)
using cached certifi-2025.9.22-py3-none-any.whl (136 kB)
using cached cryptography-44.0.0-py3-none-any.whl (1.1 MB)
using cached httpx-0.28.1-py3-none-any.whl (107 kB)
using cached idna-3.10-py3-none-any.whl (23 kB)
using cached pydantic-2.12.4-py3-none-any.whl (418 kB)
using cached pydantic-core-2.21.1-py3-none-any.whl (1.1 MB)
using cached typing-inspect-0.10.0-py3-none-any.whl (10 kB)
using cached typing-extensions-4.12.0-py3-none-any.whl (33 kB)
Installing collected packages: urllib3, typing-extensions, tenacity, sniffio, python-dotenv, pydantic, pydantic-core, pydantic, httpx, httpcore, http, cryptography, google-auth, google-genai, requests, anyio
Successfully installed anyio-4.1.2 cryptography-44.0.0 google-auth-2.36.1 google-genai-2.17.0 http-0.6.0 httpcore-1.0.5 httpx-0.28.1 idna-3.10 pydantic-2.12.4 pydantic-core-2.21.1 python-dotenv-1.2.0 requests-2.31.2 sniffio-1.3.1 tenacity-8.2.3 typing-extensions-4.12.0 typing-inspect-0.10.0 urllib3-2.2.3
(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> pip list
```

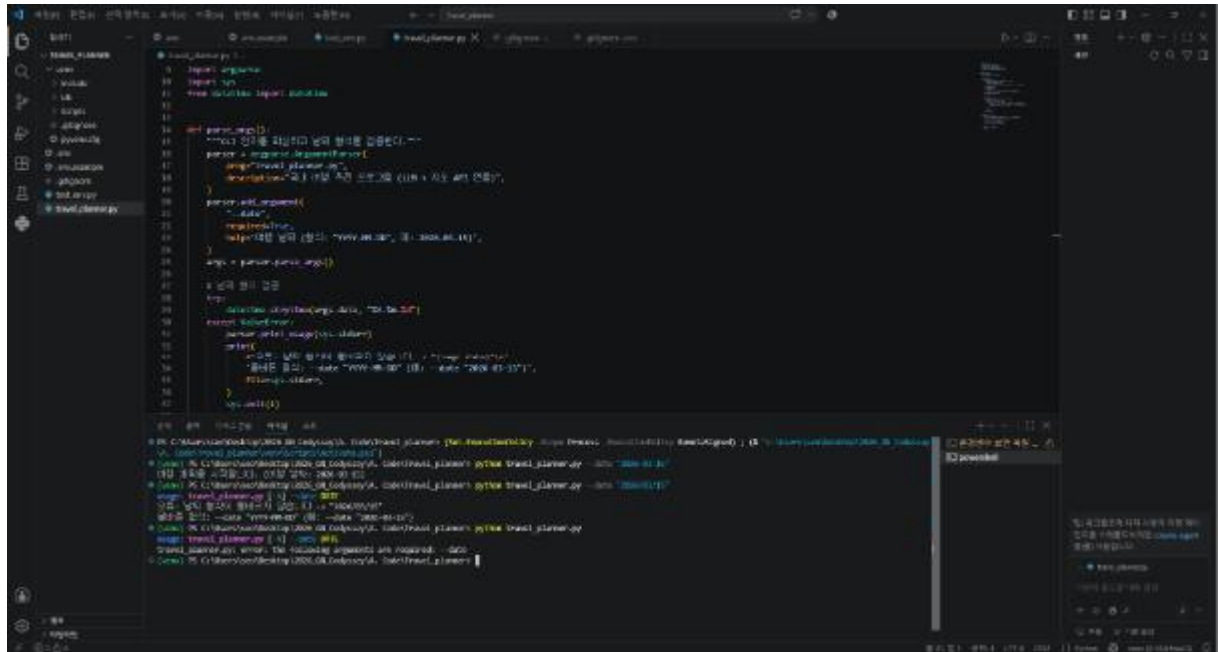
3단계. 공개 샘플 저장소 clone

[illegible]

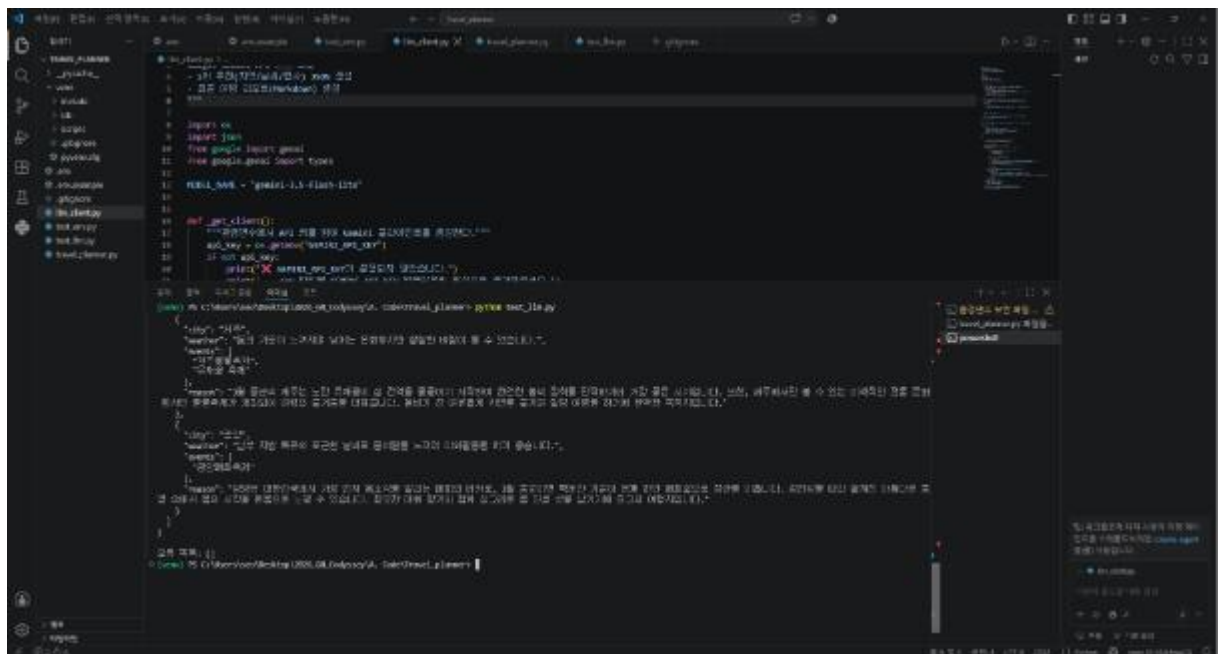
4단계. 환경변수 파일로 분리하는 보안 설정 단계(.env / .gitignore/.env.example/test_env.py)

[illegible]

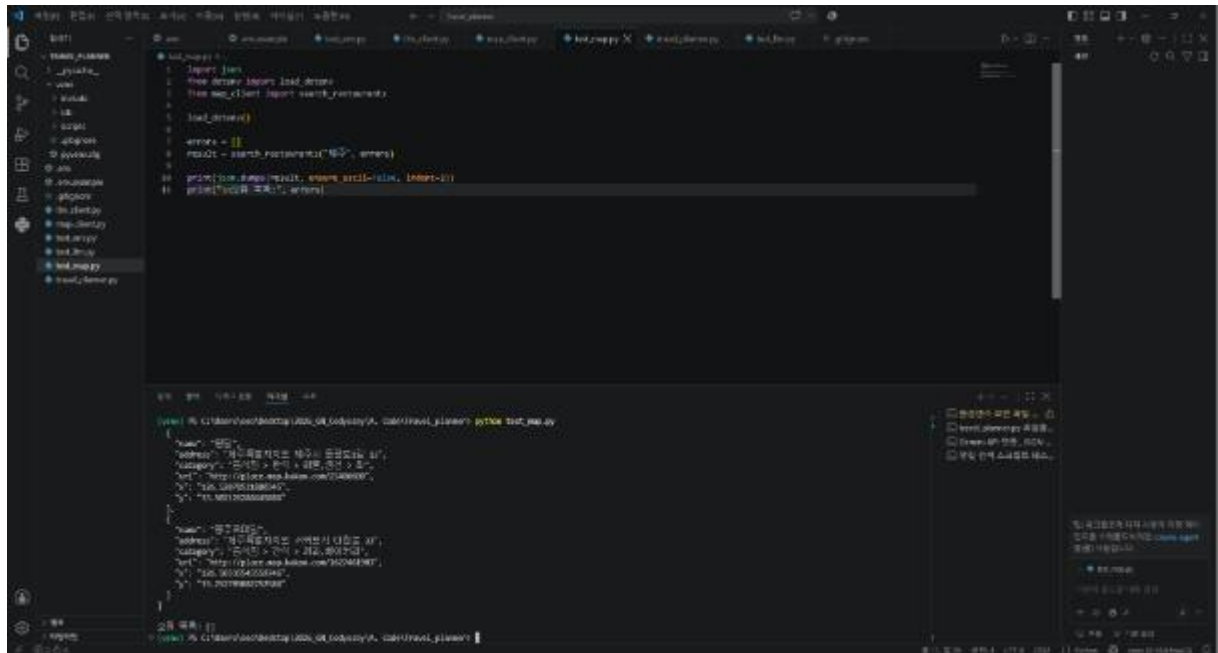
5단계. travel_planner.py 파일을 생성, 실행 테스트



6단계. Gemini API 연동_ JSON 출력_ 실행 테스트

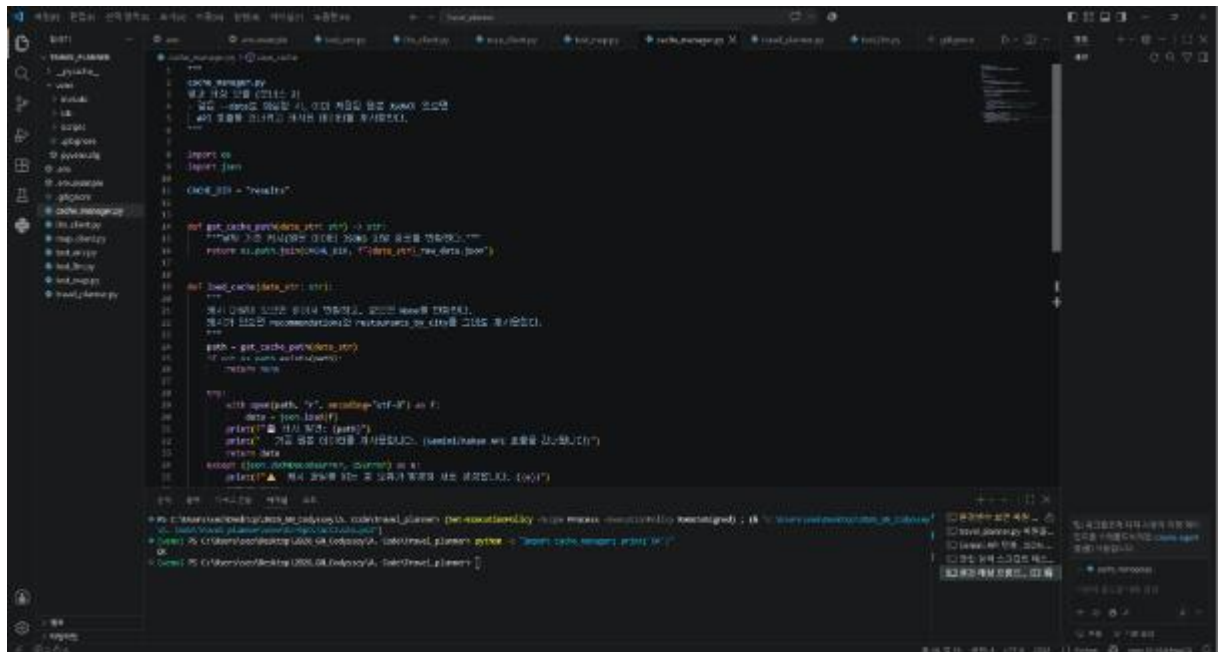


7단계. 맛집 검색 스크립트 작성 실행 테스트



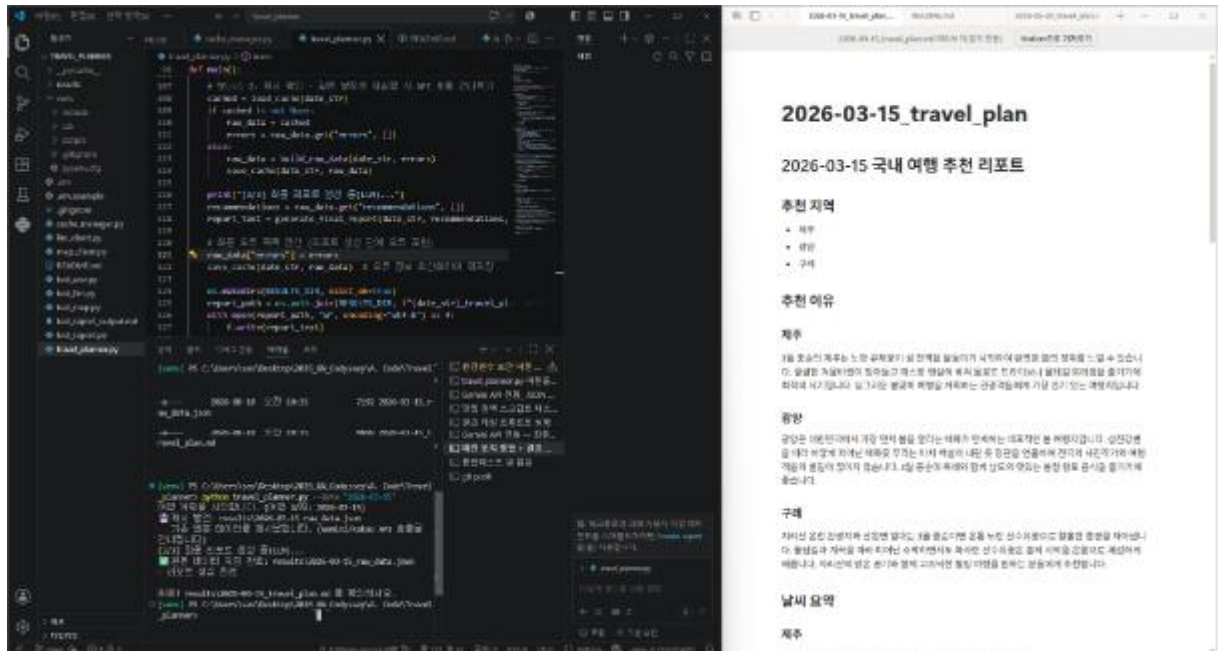
```
1 import json
2 from datetime import datetime
3 from geo_client import search_restaurants
4
5 def load_data():
6     errors = []
7     results = search_restaurants("부산", errors)
8
9     print(json.dumps(results, ensure_ascii=False, indent=2))
10    print("검색 결과 출력")
```

8단계. 결과 캐싱 _이미 저장된 1차 추천 JSON이 있으면 API 호출(Gemini)을 건너뛰고 그 데이터를 재사용

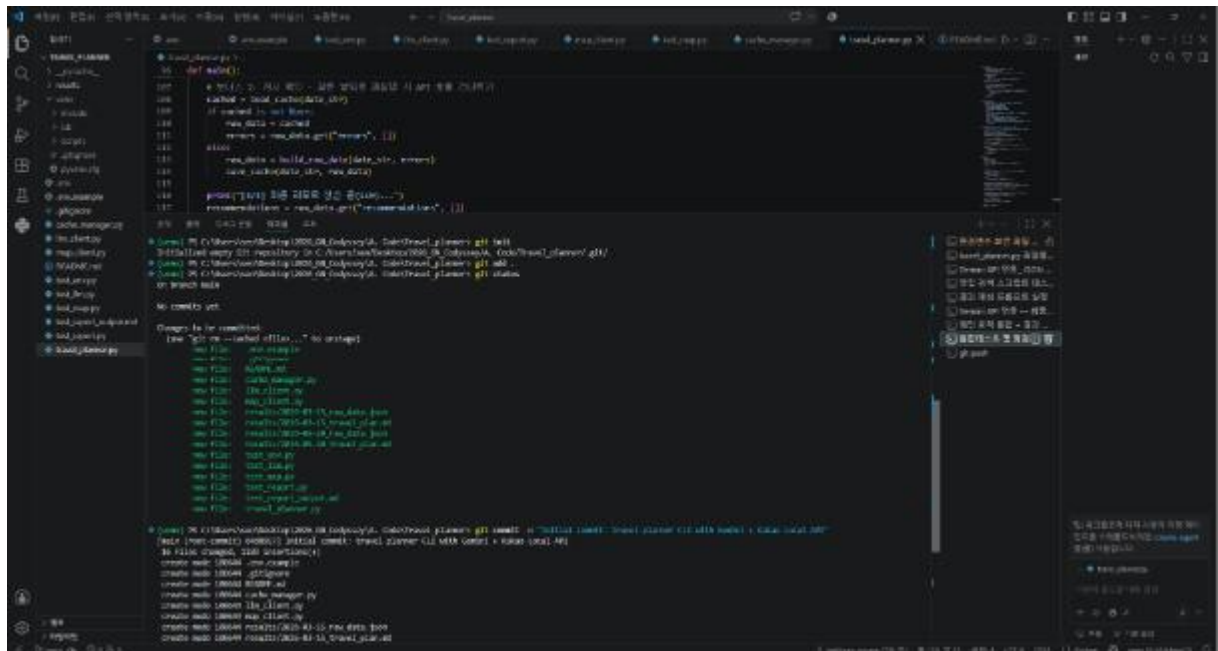


```
1 import json
2 from datetime import datetime
3 from geo_client import search_restaurants
4
5 def load_data():
6     errors = []
7     results = search_restaurants("부산", errors)
8
9     print(json.dumps(results, ensure_ascii=False, indent=2))
10    print("검색 결과 출력")
11
12 def get_cache_path(location, radius):
13     """Cache path를 생성합니다. (예: 2024-10-26_부산_10km.json)"""
14     return f"{location}_{radius}.json"
15
16 def load_cache(cache_path):
17     """캐시된 데이터를 불러옵니다. (예: 2024-10-26_부산_10km.json)"""
18     with open(cache_path, "r") as f:
19         data = json.load(f)
20     print(f"캐시된 데이터: {data}")
21     return data
22
23 def search_restaurants(location, radius, errors):
24     """주어진 위치와 반경을 기반으로 맛집을 검색합니다. (search_restaurants.py)"""
25     cache_path = get_cache_path(location, radius)
26     if not os.path.exists(cache_path):
27         # 캐시된 데이터가 없으므로 API 호출을 진행합니다.
28         results = search_restaurants(location, radius, errors)
29         with open(cache_path, "w") as f:
30             json.dump(results, f, ensure_ascii=False, indent=2)
31     else:
32         # 캐시된 데이터가 있으므로 캐시된 데이터를 사용합니다.
33         results = load_cache(cache_path)
34     return results
```

9단계. Gemini API 연동 — 최종 리포트 생성 (Markdown)



10단계. Git 보안 점검 (매우 중요)



11단계. Git push

- 1) git init
- 2) git add .
- 3) git status
- 4) git push https://github.com/bzkhseo-source/Python-Application_-Domestic-Travel-Recommendation-Program.git main

